Hybrid Vision for Drone Detection with RGB and Event Cameras

Project Report

Yajie Bin	Zhe Chen	Wenqian He	Ru Jia	Yijiao Wang	Yongdong Xie
03826243	03819300	03826593	03826148	03773922	03812671

Zhengying Xu
03812655

July 14, 2026

Motivation

Small drone detection is a challenging perception problem because drones can occupy only a few pixels, move quickly, and appear under difficult lighting and background conditions. Conventional RGB cameras provide rich appearance information, but they can suffer from motion blur and low contrast. Event cameras provide asynchronous motion information with high temporal resolution and high dynamic range, but event data alone lacks the dense texture and color information of RGB frames.

This project investigates whether a hybrid vision system that combines RGB and event-based information can improve drone detection on the FRED dataset. We implemented and compared single-modality detectors, direct RGB-event fusion models, reconstruction-based event baselines, a tracking and short-term trajectory prediction module, and a web-based system interface. The final goal was not only to train an accurate detector, but also to build an integrated and reproducible system that can be demonstrated through an API and graphical interface.

---

1 Project Description

The project addresses single-class drone detection using synchronized RGB frames and event-frame representations. The target class is `drone`. The main detector output is a bounding box, a confidence score, and a class ID. The system is extended with tracking, trajectory prediction, backend inference services, and a frontend interface for demonstration.

Goals

The project was translated into the following design goals:

- Prepare a synchronized RGB-event dataset from the FRED sequences while preserving the official train/test separation.
- Train and compare RGB-only, event-only, early-fusion, two-stream-lite, and late-fusion detection methods.
- Evaluate reconstruction-based baselines using E2VID and FireNet to test whether event-to-image reconstruction followed by YOLO is competitive with direct RGB-event fusion.
- Extend frame-level detection into temporal tracking and short-term trajectory prediction.
- Integrate the trained models into a backend API and web interface for final demonstration.

Assumptions

The central hypothesis was that RGB and event data provide complementary information. RGB frames contribute appearance and context, while event frames contribute motion-sensitive cues that can help detect small or fast-moving drones. For fusion models, we assumed that synchronized RGB-event pairs can be treated as aligned observations of the same scene if the temporal difference between both modalities is sufficiently small.

For tracking, we assumed that upstream detections are accurate enough to support frame-to-frame association. The tracker uses a near-constant-velocity assumption and IoU-based association. For trajectory prediction, we assumed that a history window of 12 frames contains enough motion context to forecast 24 future frames, corresponding to 0.8 seconds at 30 FPS.

2 Data

Sources

The project used the FRED dataset, which contains RGB frames, event-frame data, raw event information, and drone annotations. The available sequence range from 0 to 180 was used, corresponding to 181 sequences. The official split was preserved instead of creating a random split. This resulted in 144 training sequences and 37 test sequences before filtering.

Each raw sequence is stored under `raw/<sequence_id>/`. The relevant files are organized as follows:

- `RGB/`: RGB image frames.
- `Event/Frames/`: event-frame representations.
- `RGB_YOLO/`: YOLO-format drone annotations.
- Additional metadata files for sequence-level information.

Each processed sample contains an RGB image, an event-frame image, and a shared YOLO label file. Labels use the format:

```
class_id x_center y_center width height
```

All coordinates are normalized by image width and height. Since the task has only one class, the class ID is always 0.

Preprocessing

A central preprocessing step was RGB-event alignment. For each RGB frame, the pipeline selected the temporally closest event frame and stored the correspondence in:

`data/splits/official_sequences_aligned.csv`

To ensure reliable cross-modal synchronization, only pairs with an absolute timestamp difference of at most 20 ms were retained:

$$|dt| \leq 20 \text{ ms}.$$

After alignment, the dataset contained 596,327 samples before temporal filtering and 596,154 samples after the 20 ms threshold. This filtered set contained 478,812 training samples and 117,342 test samples.

The FRED sequences also contain many background frames without drone annotations. To reduce dataset size and training time while preserving localization supervision, all positive samples were kept and 10% of empty-background frames were retained using a fixed random seed. The final filtered dataset contained:

The processed dataset was stored under `FRED_ready_official_filtered/` with the following split structure:

Split	Number of synchronized samples
Training	295,663
Filtered official test / validation	71,571

Table 1: Final synchronized RGB-event dataset after temporal filtering and background-frame filtering.

- `train/images/`: RGB frames for training.
- `train/events/`: synchronized event-frame images for training.
- `train/labels/`: YOLO labels for the training split.
- `test/images/`: RGB frames for the filtered official test split.
- `test/events/`: synchronized event-frame images for the filtered official test split.
- `test/labels/`: YOLO labels for the filtered official test split.

The final processed dataset size was approximately 95 GB, with about 76 GB for training and 19 GB for test/validation. The final validation results in this report should therefore be interpreted as results on the filtered official test split, not on the unfiltered raw FRED test split.

3 System Description

The final system consists of four main layers. First, the data pipeline prepares synchronized RGB-event samples and YOLO labels. Second, the detection layer provides several single-modality and multimodal models. Third, the temporal layer converts per-frame detections into tracks and future trajectory predictions. Fourth, the engineering layer exposes the models through a backend API and web interface.

Architecture

The overall system follows this workflow:

$$\text{RGB/Event data} \rightarrow \text{Detection model} \rightarrow \text{Bounding boxes} \rightarrow \text{Tracking} \rightarrow$$
$$\text{Trajectory prediction} \rightarrow \text{Visualization/API}.$$

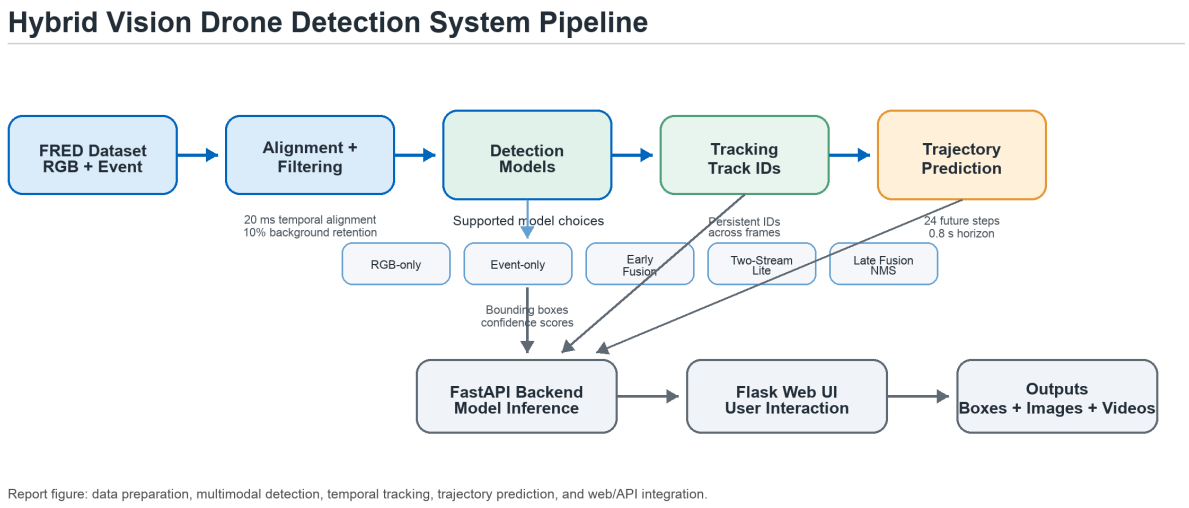


Figure 1: System Pipeline

The backend uses a unified interface for multiple detection methods. The frontend sends an RGB image, an event image, and a selected method name. The backend then loads the corresponding model weights, performs preprocessing and inference, generates visualized outputs, and returns the prediction data in JSON format.

Workflow

For detection experiments, RGB and event images are loaded from the synchronized dataset. RGB-only and event-only models use one modality. Early fusion and two-stream-lite use paired RGB-event data. Late fusion first runs two separate detectors and then merges their prediction boxes.

For tracking and prediction, the detection outputs are passed to a tracker that assigns persistent IDs across frames. Confirmed tracks are then used by a Transformer-based predictor to forecast future bounding boxes. The web interface allows users to upload or select synchronized inputs, choose a model, start detection, and inspect output images or videos.

Models

RGB-only and Event-only Detectors

The RGB-only and event-only baselines use Ultralytics YOLOv8 detectors (Jocher et al., 2023) trained on a single modality. The RGB-only model receives RGB frames, while the event-only model receives event-frame images exposed under a YOLO-compatible `images/` directory. These two models are also used as the basis for late fusion.

Late Fusion with NMS

Late fusion is a decision-level baseline. It trains one RGB detector and one event detector independently. At inference time, both detectors produce boxes in the same format:

```
filename x1 y1 x2 y2 confidence class_id
```

The two prediction sets are merged with Non-Maximum Suppression (NMS) (Neubeck and Van Gool, 2006) using an IoU threshold of 0.45. A Weighted Boxes Fusion variant (Solovyev et al., 2021) was also tested, but its official evaluation result matched that of NMS, so the final report uses the simpler NMS version. Late fusion is easy to reproduce and does not require architectural changes, but it can only combine modalities after detection and cannot learn joint RGB-event features during training.

Early Fusion with 6-channel Input

Early fusion combines RGB and event data at the input level. During training, each RGB-event pair is converted to a 6-channel TIFF image:

$$[R, G, B, E_R, E_G, E_B].$$

If the event image size does not match the RGB image size, the event image is resized before stacking. The first convolution layer of YOLOv8 was modified from 3 input channels to 6 input channels. RGB weights were copied from the pretrained YOLOv8 model, while the event-channel weights were initialized separately using Kaiming initialization (He et al., 2015) in the selected setting.

Two-Stream-Lite Fusion

Two-stream-lite is a shallow feature-level fusion model. It uses a custom RGB-event stem at the beginning of a YOLOv8m-style detector:

$$\begin{aligned} \text{RGB} &\rightarrow \text{Conv}(3 \rightarrow c/2) \\ \text{Event} &\rightarrow \text{Conv}(3 \rightarrow c/2) \\ &\rightarrow \text{Concat} \rightarrow 1 \times 1 \text{ Conv} \rightarrow \text{YOLOv8m backbone/head.} \end{aligned}$$

This design keeps modality-specific features separate for the first processing stage and fuses them before the main backbone. It adds only a small custom stem while keeping most of the detector unchanged.

Reconstruction-based Baselines

To compare direct fusion against reconstruction-based approaches, two event video reconstruction methods were evaluated: E2VID and FireNet. Raw event data were reconstructed into intensity-like images, and then the Team A event-domain YOLO detector was applied. This baseline tests whether event reconstruction followed by standard detection is sufficient for the FRED drone detection task.

Tracking and Trajectory Prediction

The tracking module follows a ByteTrack-style design. It uses a Kalman filter with a constant-velocity motion model and a two-stage association strategy. High-confidence detections are matched first; low-confidence detections can then recover temporarily lost tracks. The output format is:

```
timestamp,track_id,x,y,w,h
```

The trajectory predictor uses a Transformer encoder-decoder. It observes 12 past frames and predicts 24 future bounding boxes. Event crops are processed by a lightweight CNN and fused with normalized bounding-box coordinates before being passed to the Transformer. The prediction output format is:

```
ts_obs_end track_id step pred_x1 pred_y1 pred_x2 pred_y2
```

Tools

The detection models are based on Ultralytics YOLOv8 (Jocher et al., 2023). Custom fusion components were implemented in Python and PyTorch. The tracking and prediction modules use a custom ByteTrack-style tracker and a PyTorch Transformer predictor. The backend is implemented with FastAPI and prepared for Docker deployment. The frontend is implemented with Flask, HTML, CSS, and JavaScript.

4 Results

Detection Results

The main detection metrics are Precision, Recall, and AP50. Since the project has only one class, AP50 is equivalent to mAP50 at IoU 0.5.

Model	Precision	Recall	AP50	Output logic
RGB-only	0.721	0.466	0.444	RGB YOLO
Event-only	0.644	0.542	0.502	Event YOLO
Early Fusion	0.816	0.787	0.777	6-channel input fusion
Two-Stream-Lite	0.827	0.787	0.786	Two stems + feature fusion
Late Fusion NMS	0.662	0.869	0.739	Two detectors + NMS

Table 2: Detection performance on the filtered official test split.

The event-only model outperforms the RGB-only model in AP50 and recall, indicating that event frames provide useful motion-related information. Early fusion improves substantially over both single-modality baselines. Two-stream-lite achieves the best AP50 and precision, while late fusion achieves the highest recall but lower precision. Overall, two-stream-lite is selected as the best balanced direct fusion model.

Training Runtime and Computational Cost

Model	Architecture	Img size	Batch	Observed epochs	Runtime
RGB-only	YOLOv8n	640	128	30	05:50:27
Event-only	YOLOv8n	640	128	30	05:48:16
Late Fusion NMS	RGB YOLOv8n + Event YOLOv8n	640	128	30 + 30	11:38:43
Early Fusion	YOLOv8m, 6-channel input	1280	16	7	12:09:41
Two-Stream-Lite	YOLOv8m, custom stem	1280	16	9	14:00:24

Table 3: Training configuration and runtime summary. Runtime for continued runs is reported as the selected training chain.

Late fusion itself has no separate training stage; the reported runtime is the sum of the RGB-only and event-only detector training runs.

The available training logs show that early fusion with image size 1280, batch size 16, and AMP enabled reached a peak GPU memory usage of approximately 25.5 GB on an NVIDIA RTX 5090 D with 32 GB memory. An early-fusion validation log reported approximately 4.5 ms inference time per image. Late fusion requires two detector passes and an additional NMS step at inference time; however, its two sub-models are YOLOv8n detectors at image size 640, while early fusion and two-stream-lite use YOLOv8m-style models at image size 1280. Therefore, the exact latency ranking should be interpreted cautiously until all models are benchmarked under the same hardware, batch size, and measurement script.

Reconstruction Baseline Results

Method	Images	Precision	Recall	AP50	mAP50-95
E2VID + YOLO	1,602	0.000438	0.00528	0.00000421	0.00000171
FireNet + YOLO	8,790	0.221	0.0797	0.0363	0.00846

Table 4: Reconstruction-based event baselines.

Detection coverage was also computed as the fraction of reconstructed frames with at least one YOLO detection. E2VID produced detections in 64 of 1,759 reconstructed frames, corresponding to 3.64% coverage. FireNet produced detections in 232 of 10,321 reconstructed frames, corresponding to 2.25% coverage. Although FireNet performed better than E2VID in AP50, both reconstruction-based baselines remained far below direct RGB-event fusion.

Tracking and Prediction Results

The tracking and prediction module was evaluated on sequence 51 using the output of the detection pipeline. Tracking was evaluated with an IoU threshold of 0.5. The trajectory predictor forecasts 24 future steps, corresponding to 0.8 seconds at 30 FPS.

Tracking metric	Value
Precision	0.8327
Recall	0.8256
F1	0.8292
MOTA	0.6562
MOTP	0.7130
ID Switches	15
TP / FP / FN	3504 / 704 / 740

Table 5: Tracking evaluation results from `evaluate.sh`.

Prediction metric	Value
Matched predictions	71,169
Trajectories	2,968
ADE	50.82 px
FDE	92.12 px

Table 6: Trajectory prediction results from `evaluate.sh`.

The prediction error increases with the forecast horizon. The mean center-distance error is 26.52 px at 0.1 s, 45.17 px at 0.4 s, and 91.56 px at 0.8 s. This trend is expected because long-term forecasts are more uncertain than short-term forecasts.



Figure 2: Qualitative tracking and trajectory prediction example. The current tracked drone position is shown together with predicted future bounding boxes; the future boxes progress from near-term predictions to farther-term predictions.

System Integration Results

The backend exposes a unified API for `early_fusion`, `two_fusion`, and `late_fusion`. Each method accepts synchronized RGB and event inputs and returns bounding boxes, confidence scores, class labels, and visualization paths. The Docker configuration packages the required model weights and exposes the same API endpoints in a reproducible environment.

The web interface provides an interaction layer for model selection, file upload, detection execution, and visualization. It supports both individual RGB-event image pairs and synchronized frame sequences stored in ZIP files. For sequence input, the system generates RGB input, event input, and detection output videos so that users can compare the streams side by side.

Observations

The results show three main observations. First, event information is useful for drone detection: the event-only model has higher recall and AP50 than the RGB-only model. Second, direct RGB-event fusion is more effective than reconstruction-based detection. Third, two-stream-lite provides the best balance between AP50 and precision, while late fusion is useful when high recall is preferred.

Trends

The main performance trend is:

$$\text{RGB/Event direct fusion} \gg \text{reconstruction baseline.}$$

Within direct fusion, two-stream-lite slightly outperforms early fusion in AP50, suggesting that preserving modality-specific early features before fusion is beneficial. Late fusion increases recall but introduces more false positives because it combines outputs from two detectors. In trajectory prediction, error increases over time, which is consistent with the growing uncertainty of future motion.

5 Discussion

Interpretation of the Results

The detection results support the main project hypothesis: RGB and event data are complementary for drone detection. Single-modality RGB detection is limited by the small size and low contrast of drones in some frames. Event-only detection improves recall because event frames emphasize motion. However, direct fusion performs best because it allows the detector to combine appearance and motion cues.

Two-stream-lite is the most balanced model. Its feature-level fusion is more expressive than output-level late fusion and more structured than pure input-level stacking. Late fusion remains useful as an interpretable baseline and achieves the highest recall, but it requires two detectors and is less precise. The reconstruction baseline shows that reconstructing intensity images from events does not preserve small drone details reliably enough for strong YOLO detection on this dataset.

The tracking module demonstrates that per-frame detections can be converted into temporally consistent tracks. With a tracking precision of 0.8327, recall of 0.8256, and 15 ID switches, the tracker provides a usable temporal extension of the detection pipeline. The trajectory predictor gives reasonable short-term forecasts, but its error grows with the prediction horizon, reaching an FDE of 92.12 px at 0.8 s.

Critical Assessment of the Results and Assumptions

Several assumptions were only partially satisfied. First, RGB-event synchronization was controlled by a 20 ms threshold, but the filtered dataset is not necessarily temporally continuous. This is suitable for detection evaluation but less ideal for tracking and prediction tasks that require continuous frame windows.

Second, direct comparison between all models depends on consistent evaluation splits. The final report uses the filtered official test split as the main evaluation basis. Any small validation subset results were treated only as demonstration results and not as the final model ranking.

Third, the tracking and prediction modules depend strongly on upstream detection quality. The tracker cannot recover objects that were never detected, and the predictor can be affected by fragmented or noisy tracks. The predictor was trained on cleaner ground-truth tracks, while inference uses tracker-generated tracks. This distribution mismatch likely limits prediction quality.

Finally, computational cost was not logged uniformly for every model. We have reliable training runtimes and early-fusion GPU memory information, but peak memory and inference speed are incomplete for some models. A more complete resource benchmark would require running all models under the same hardware and measurement script.

6 Conclusion

This project built a hybrid RGB-event drone detection system on the FRED dataset. The work included synchronized data preparation, single-modality baselines, RGB-event fusion models, reconstruction baselines, tracking and trajectory prediction, and frontend-backend integration.

Summary of the Results

The final filtered dataset contains 295,663 training samples and 71,571 filtered official test samples. Among detection models, two-stream-lite achieved the best AP50 of 0.786 and the best precision of 0.827. Early fusion also improved clearly over RGB-only and event-only baselines. Late fusion achieved the highest recall of 0.869 but required two detectors and had lower precision. Reconstruction-based baselines using E2VID and FireNet performed much worse than direct fusion, supporting the decision to use RGB-event fusion rather than event reconstruction followed by YOLO.

The tracking module achieved 0.8327 precision, 0.8256 recall, 0.6562 MOTA, and 15 ID switches on sequence 51. The trajectory prediction module achieved an ADE of 50.82 px and an FDE of 92.12 px. These results show that the system can extend detection outputs into temporal tracks and short-term future predictions, although prediction uncertainty increases with time.

Future Work

Future work should include a uniform resource benchmark for all models, including inference time, FPS, and peak GPU memory under the same hardware conditions. The two-stream-lite model could also be further optimized to reduce runtime while preserving accuracy.

For tracking and prediction, future work should train the predictor on tracker-like noisy inputs rather than only clean ground-truth tracks. A constant-velocity baseline should be added to quantify whether the Transformer predictor improves over simple motion extrapolation. Finally, continuous temporal sequence exports would make tracking and forecasting evaluation more reliable than using a detection-oriented filtered dataset.

References

G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” Ultralytics, 2023. [Online]. Available: `https://github.com/ultralytics/ultralytics`. Accessed: Jul. 14, 2026.

K. He, X. Zhang, S. Ren, and J. Sun, “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1026–1034, doi: 10.1109/ICCV.2015.123.

R. Solovyev, W. Wang, and T. Gabruseva, “Weighted Boxes Fusion: Ensembling Boxes from Different Object Detection Models,” *Image and Vision Computing*, vol. 107, Art. no. 104117, 2021, doi: 10.1016/j.imavis.2021.104117.

A. Neubeck and L. Van Gool, “Efficient Non-Maximum Suppression,” in *Proceedings of the 18th International Conference on Pattern Recognition (ICPR)*, vol. 3, 2006, pp. 850–855, doi: 10.1109/ICPR.2006.479.

7 Comments to the Group Work Experience

The project required close coordination between data preparation, model training, reconstruction baselines, tracking, backend deployment, frontend visualization, and final evaluation. A major challenge was that different components used different file formats, output folders, and evaluation assumptions. The final integration work therefore focused on aligning model inputs and outputs, clarifying the evaluation split, and presenting the results with a consistent technical narrative.

The collaboration also showed the importance of reproducible logging. Some components had complete metrics and runtime records, while others required additional clarification or re-evaluation. For future projects, we would define common output formats, metric scripts, and logging conventions earlier in the development process.